Sumup - RO

Bayes Filter

Prediction and Correction Step

- Bayes filter written as 2-Step process
  - $bel(x_t) = \eta \cdot p(z_t|x_t) \qquad \int p(x_t|x_{t-1}, u_t) \cdot bel(x_{t-1}))dx_{t-1}$
- **Prediction step**
  - $\overline{bel}(x_t) = \int \underbrace{p(x_t|x_{t-1}, u_t)}_{\text{motion model}} \cdot bel(x_{t-1}))dx_{t-1}$
- **Correction step**
  - $bel(x_t) = \eta \cdot \underbrace{p(z_t|x_t)}_{\text{observation model}} \overline{bel}(x_t)$
  - Observation/ measurement/ sensor model

Occupancy Grid Models

- Compute **most likely map** given
  - the sensor data
  - the robot's poses ($x_{1:t}$) instead of the sensor controls ($u_{1:t}$)
- **Disadvantage**: Memory (especially 3D)
- **for noisy wide-angle sonar sensors**
- "*Mapping with known poses*"

Formal Assumptions

1. Cell area either occupied or completely free
   - **Binary Random Variable** modelling occupany of its space
   - $P(M_i = occ) = P_{occ}(M_i) = p(m_i) = 1$
   - $P(M_i = free) = P_{free}(M_i) = 1 - P_{occ}(M_i) = p(\neg m_i) = 0$
   - No knowledge/prior: $p(m_i) = 0.5$
2. World is **static** (Like most mapping systems)
3. Grid cells are **independent** of each other
   - -> $\underset{map}{p(m)} = \prod_i \underset{\text{cell}}{p}(m_i)$

- Log odds ratio defined as
  - $l(x) = \log \frac{p(x)}{1-p(x)}$

- and with p(x)
  - $p(x) = 1 - \frac{1}{1+\exp l(x)}$

Occupancy Mapping in Log Odds Form

- Products turn into a sum

$$l(m_i|z_{1:t}, x_{1:t}) = \underbrace{l(m_i|z_t, x_t)}_{\text{inverse sensor model}} + \underbrace{l(m_i|z_{z:t-1}, x_{1:t-1})}_{\text{recursive term}} - \underbrace{l(m_i)}_{prior}$$

- or in short
  - $l_{t,i} = inv_sensor_model(m_i, x_t, z_t)) + l_{t-1,i} - l_0$

Locomotion

Differential Drive Robot

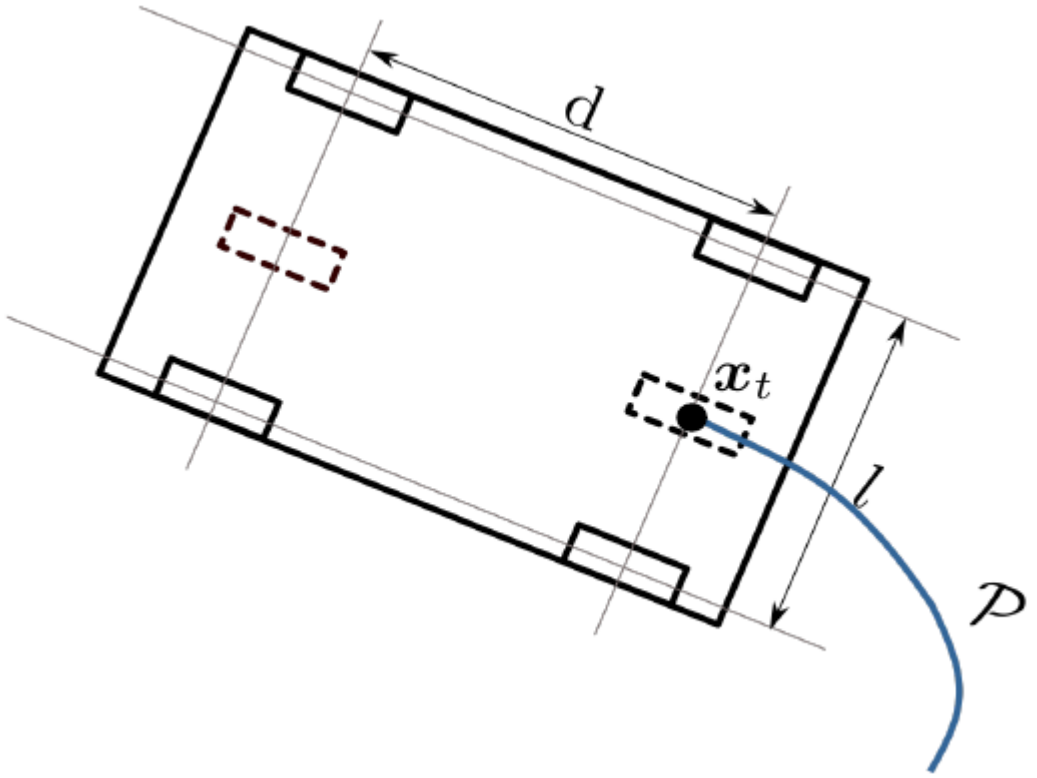
- Unknowns:
  - translation velocity $v = \frac{(v_r+v_l)}{2}$
  - rotational velocity $\omega = \frac{(v_r-v_l)}{l}$
  - radius of curve $r = \frac{l}{2}\frac{(v_r+v_l)}{(v_r-v_l)}$

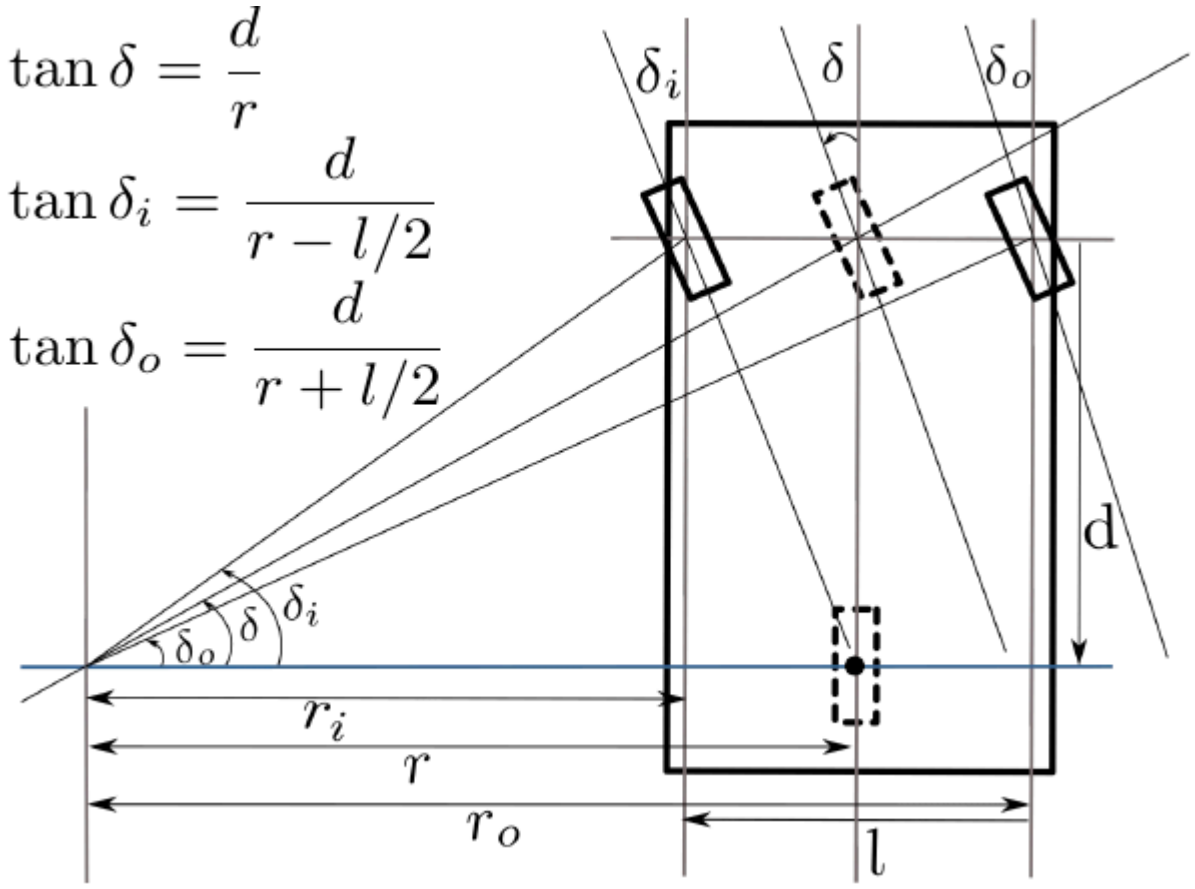
ICC - Instantaneous Center of Curvature

$$\boldsymbol{x}_{ICC} = \begin{bmatrix} x - r\sin\theta \\ y + r\cos\theta \end{bmatrix}$$

Ackermann Drive

- E.g. for modelling a car

- 

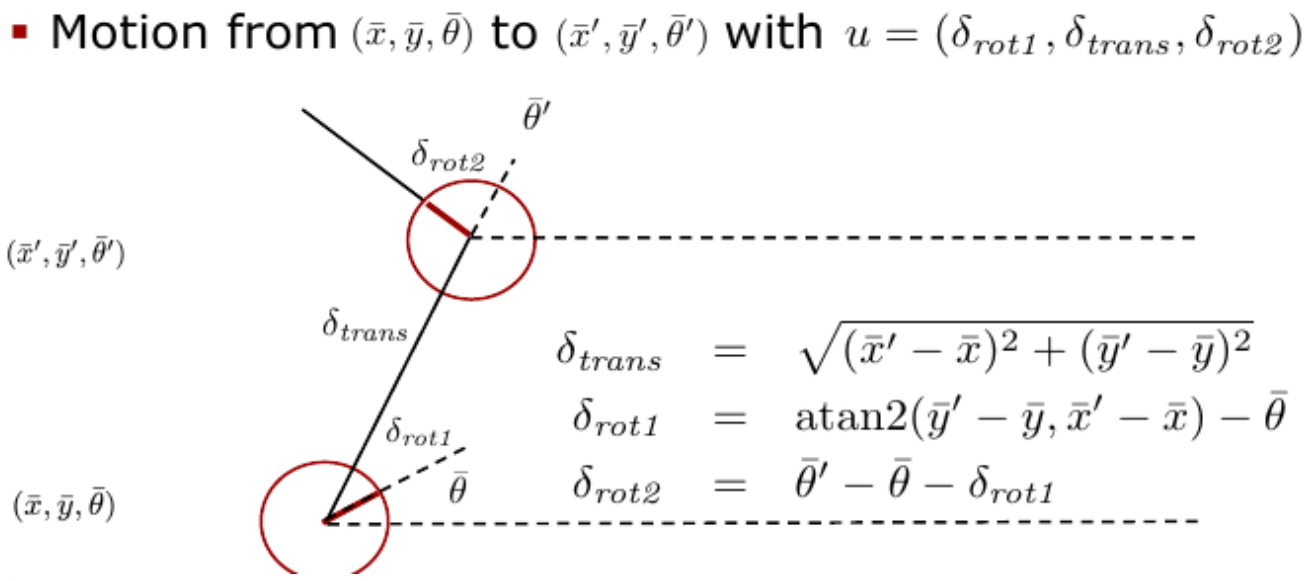
- 

Typical Motion Models

Motion Estimation is Noisy!

- Model parameters have to be learnt/estimated
- Calculating posterior probability $p(x_t|u_t, x_{t-1})$

- Sampling from $p(x_t|u_t, x_{t-1})$
- 2 common types:
- **Velocity-based (Dead reckoning)**
  - Pose calculation from (velocities and elapsed time)
  - Dead Reckoning from "deduced reckoning"
  - Historically used to log position of ships
    - Typically Wheel Encoders (visual black and white -> 0/5V)
- Noise Model: True motion corrupted with noise
    - $\hat{\nu} = \nu + \epsilon_{\alpha_1|\nu|+\alpha_2|\omega|}$
    - $\hat{\omega} = \omega + epsilon_{\alpha_3|\nu|+\alpha_4|\omega|}$
    - $\hat{\gamma} = \epsilon_{\alpha_5|\nu|+\alpha_6|\omega|}$
- $r^{ICC} = \sqrt{(x^{ICC} - x)^2 + (y^{ICC} - y)^2}$
- $\Delta\theta = atan2(y' - y^{ICC}, x' - x^{ICC}) - atan2(y - y^{ICC}, x - x^{ICC})$
- Compute velocities:
    - $v = \frac{\Delta\theta}{\Delta t} r^{ICC}$
    - $w = \frac{\Delta\theta}{\Delta t}$
-
- **Odometry-based**
  - Pose calculation from **wheel encoders** values
  -



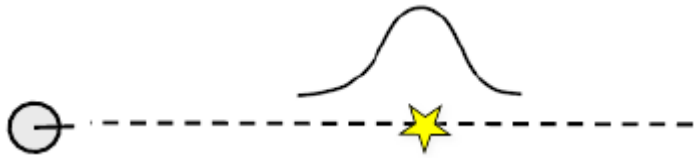
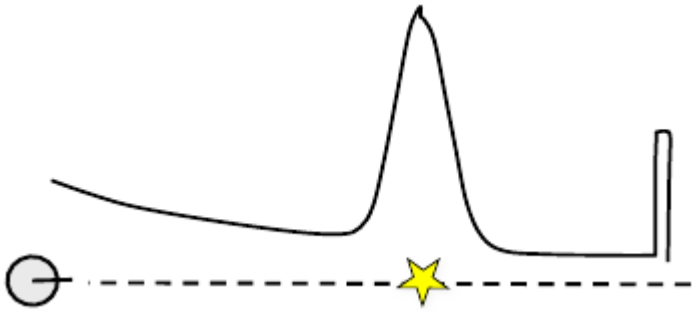
  - δ's are with uncertainty/noise
  - Difference between odometry and hypothesis

$$p_1 = \text{prob}(\delta_{\text{rot1}} - \delta_{\text{rot1}}, \alpha_1 |\delta_{\text{rot1}}| + \alpha_2 \delta_{\text{trans}})$$
$$p_2 = \text{prob}(\delta_{\text{trans}} - \hat{\delta}_{\text{trans}}, \alpha_3 \delta_{\text{trans}} + \alpha_4 (|\delta_{\text{rot1}}| + |\delta_{\text{rot2}}|))$$
$$p_3 = \text{prob}(\delta_{\text{rot2}} - \hat{\delta}_{\text{rot2}}, \alpha_1 |\delta_{\text{rot2}}| + \alpha_2 \delta_{\text{trans}})$$

  -

- Gaussian or sampling!
- Typical banana-shaped distributions obtained for the 2D-projection of the 3D posteriors

Ray-Cast Model

- **Simple** Ray-cast models consider **first obstacle** along LoS
  - Gaussian noise in the distance
  - 
- **More Advanced** Ray-Cast Model:
  - **Mixture** of 4 models
    - 
  - **Dynamic objects**
    - Account for closest obstacle in a uniform distribution of objects: Logarithmic
    - Given obstacle find bit strings from random bit string
      - 1*** -> 8
      - 01** -> 4
      - 001* -> 2
      - 0001 -> 1
      - $P_{unexp}(z|x,m) = \lambda \exp(-\lambda z) \quad z < z_{exp}, 0 \text{ otherwise}$
  - **Random**
    - constant
  - **Max-range**
    - constant peak at z_{max}
  - **Noise**
- Similar Results for Lasers and Sonars

Landmark Observation

- Raycasting models for range sensors

- Beam-endpoint models for range sensors
- Simple models for feature/point observations:
  - triangulation
  - range-only
  - bearing-only

Simple Range Bearing Sensors

- 2 Cameras: Stereo Cameras Measure Point in Space via Triangulation
- Model for perceiving Landmarks in 2D from
  - Pose (x, y, θ)
- Expected observation of feature j at location $(m_{j,x}, m_{j,y})^T$
  - $z_t^i = \begin{bmatrix} r_t^i \\ \phi_t^i \end{bmatrix} = \begin{bmatrix} \sqrt{(m_{j,x} - x)^2 + (m_{j,y} - y)^2} \\ atan2(m_{j,y} - y, m_{j,x} - x) - \theta \end{bmatrix}$
  - Gaussian noise with covariance Q_t in range and bearing measurement

Bearing-Only sensors

- Bearing $z_t^i = \phi_t^i$
  - $\phi_t^i = atan2(m_{j,y} - y, m_{j,x} - x) - \theta$

Mapping of 3D Point to Pixel Coordinate

$$\underbrace{\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}}_{\text{pixel coordinate}} = \underset{\text{transformation}}{P} \underbrace{\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}}_{\text{world coordinate}}$$

Where $P = KR[\boldsymbol{I}_3 | -\boldsymbol{X}_O]$

$$\underbrace{\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}}_{\text{2D pixel coordinate}} = \lambda \underbrace{\begin{bmatrix} c & cs & x_H \\ 0 & c(1+m) & y_H \\ 0 & 0 & 1 \end{bmatrix}}_{\text{camera calibration matrix}} \underbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}}_{\text{rotation matrix}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & -X_O \\ 0 & 1 & 0 & -Y_O \\ 0 & 0 & 1 & -Z_O \end{bmatrix}}_{\text{projection matrix and translation}} \underbrace{\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}}_{\text{3D world coordinate}}$$

DLT: Direct Linear Transform

- Homogeneous Projection Matrix
  - $P = KR[\boldsymbol{I}_3 | -\boldsymbol{X}_O]$
- Contains **11 parameters**
  - 6 **extrinsic parameters**: $R, \boldsymbol{X}_O$

- 5 **intrinsic parameters**: c, x_H, y_H, m, s
- Mapping form 3D point to 2D pixel coordinate: $\boldsymbol{x} = p\boldsymbol{X}$
- **Inverse Mapping**
  - λ: factor resulting from H.C.
  - $\boldsymbol{X} = \boldsymbol{X}_O + \lambda(KR)^{-1}\boldsymbol{x}$
    - Describes line in 3D world
    - $\lambda(KR)^{-1}\boldsymbol{x}$ describes direction of ray from Camera origin $\boldsymbol{X}_O$ to 3D poing $\boldsymbol{X}$
    - Measuring directions

Pose Estimation From Scan Matching

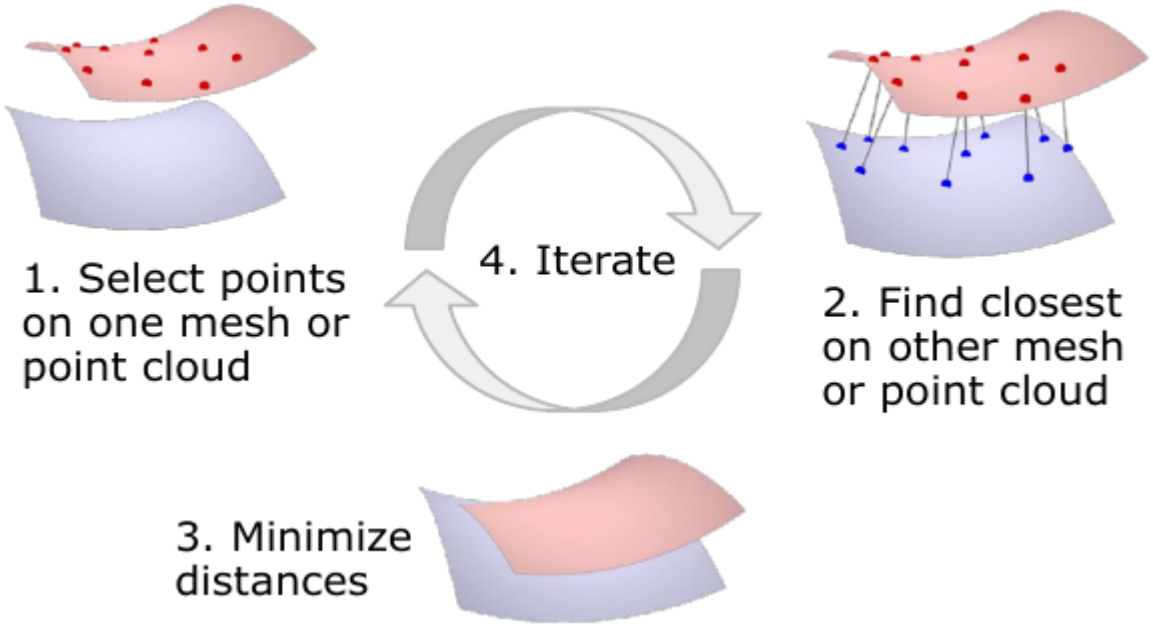
- Movement - Each scan yields point cloud
- With knowledge on correspondence between these clouds: Alignment via Least Squares Minimisation

$$\hat{\boldsymbol{x}}_{\boldsymbol{n}} = R\boldsymbol{x}_n + \boldsymbol{t}$$
$$\sum ||\boldsymbol{y}_n - \hat{\boldsymbol{x}}_{\boldsymbol{n}}||^2 \rightarrow min$$

- Usage of singular value decomposition
- Point Cloud Registration with **Unknown Data Associations**
  - No direct and optimal solution exists

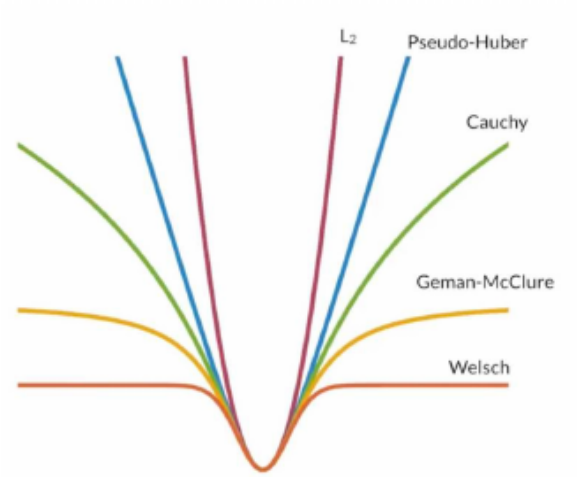
Iterative Closest Point (ICP)

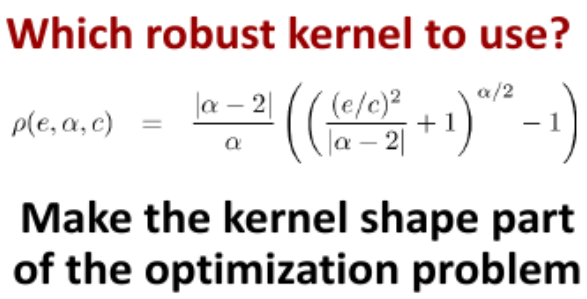
- If correct correspondance **unknown** -> impossible to determine **optimal parameters**
  - **Iteratively estimate data association and transformation given the data association**
    1. Pick for every point its closest neighbor in the other point cloud ("closest point")
    2. Compute rigid body transform & align clouds
    3. Repeat
  - If initial point clouds/correspondences are "close enough" -> Convergence



KISS - ICP - Keep It Small Simple

- **Motion prediction and scan de-skewing**
  - Constant velocity model de-skewing and prediction
  - Registration to correct deviation from the prediction
  - Possible extension: integrate IMU / VO / ...
- **Point cloud subsampling**
  - Voxel grid subsampling of each scan for registration
- **Local map and correspondence estimation**
  - Sliding voxel grid as local map
  - Register downsampled scan against local map
  - Maintian only N(=20) scans per voxel
  - Adaptive correspondence search based on statistics of registrations done so far
- **Alignment through robust optimisation**
  - Least squares with Gmean-McClure kernel
    - or even more fancy robust optimisation
- Adaptive Threshold
- Limit Search Area of Correspondences
- Outlier Handling with Robust Cost Kernels - e.g. of moving elements
  - Outlier rejection in presence of dynamic objects
  - Relax least squares formulation
  - Downweigh potential outliers

- 



- (Fast) Semantic Segmentation on Range Images

Summary

- Estimate which parts of scene are static and which are dynamic (heuristics, learning-based, ...)
- Mark point belonging to dynamic objects
- Perform an informed motion prediction
- Compute a good initial guess of the data association
- Run ICP with outlier rejection & robust optimisation

Kalman Filter

- **Bayes filter** for the **Gaussian linear case**
  - Every distribution is Gaussian
  - $p(x) = det(2\pi\Sigma)^{-\frac{1}{2}} exp(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu))$
  - Given:
  - $x = \begin{bmatrix} x_a \\ x_b \end{bmatrix}$
  - $p(x) = \mathcal{N}$
- Marginals are Gaussians
  - $p(x_a) = \mathcal{N}$
  - $p(x_b) = \mathcal{N}$
- Conditionals are Gaussians
  - $p(x_a|x_b) = \mathcal{N}$
  - $p(x_b|x_a) = \mathcal{N}$
- Given $p(x) = p(x_a, x_b) = \mathcal{N}(\mu, \Sigma)$ with:
  - $\mu = \begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix}$

- $\Sigma = \begin{bmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{bmatrix}$

- The **marginalisation distribution** is
  - $p(x_a) = \int p(x_a, x_b) dx_b = \mathcal{N}(\mu, \Sigma)$
    - with $\mu = \mu_a$ and $\Sigma = \Sigma_{aa}$
- The **conditional distribution** is:
  - $p(x_a|x_b) = \frac{p(x_a, x_b)}{p(x_b)} \mathcal{N}(\mu, \Sigma)$
    - with $\mu = \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(x_b - \mu_b)$
    - $\Sigma = \Sigma_{aa} - \Sigma_{ab}\Sigma bb^{-1}\Sigma_{ba}$
- Optimal solution for linear models and Gaussian distributions
- Performs recursive state estimation
- Prediction step to exploit the controls
- Correction step to exploit the observations

$$bel(x_t) = \eta \cdot \underbrace{p(z_t|x_t)}_{correction(observation)} \int \underbrace{p(x_t|x_{t-1}, u_t)}_{prediction(control)} \cdot \underbrace{bel(x_{t-1})}_{\text{recursive belief}} dx_{t-1}$$

- ϵ_t & δ_t: Random variables representing process and measurement noise
  - assumed to be independent and normally distributed with covariance R_t and Q_t respectively
-
  - Product of 2 Gaussians is a Gaussian
- Gaussians stay Gaussians
  - under linear transformations
  - Marginal and conditional distribution of a Gaussiann stay
- Computing mean and covariance of marginal and conditional of a Gaussian
- Matrix inversion lemma
- ## Useless/Perfect Sensor (Q_t)
- Infinite noise: $Q_t = diag(\infty)$
  - Leads to:

$$4: \quad K_t = \bar{\Sigma}_t\, C_t^\top (\cancel{C_t\, \bar{\Sigma}_t\, C_t^\top + Q_t})^{-1} \quad \to 0$$
$$5: \quad \mu_t = \bar{\mu}_t + K_t(z_t - C_t\, \bar{\mu}_t)$$

$$5: \quad \mu_t = \bar{\mu}_t + \underbrace{K_t(z_t - C_t\, \bar{\mu}_t)}_{\to 0}$$

- $$5: \quad \mu_t = \bar{\mu}_t$$
  - **Oservation** will be **ignored**

- No noise: $Q_t = diag(0)$
  - Leads to

$$4: \quad K_t = \bar{\Sigma}_t\, C_t^\top (C_t\, \bar{\Sigma}_t\, C_t^\top + \cancel{Q_t})^{-1} \quad \to C_t^{-1}$$
$$5: \quad \mu_t = \bar{\mu}_t + K_t(z_t - C_t\, \bar{\mu}_t)$$

$$5: \quad \mu_t = \bar{\mu}_t + K_t(z_t - C_t\, \bar{\mu}_t)$$
$$\to C_t^{-1} z_t - \bar{\mu}_t$$

- $$5: \quad \mu_t = C_t^{-1} z_t$$
  - **Prediction** will be **ignored**

KF Assumptions

- Gaussian distributions and noise
- Linear motion and observation model
- $x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$
- $z_t = C_t x_t + \delta_t$

Extended Kalman Filter

EKF Linearisation

- Jacobian matrix

- It is a non-square matrix $m \times n$ in general
- Given a vector-valued function

$$g(x) = \begin{pmatrix} g_1(x) \\ g_2(x) \\ \vdots \\ g_m(x) \end{pmatrix}$$

- The Jacobian is defined as

$$G_x = \begin{pmatrix} \frac{\partial g_1}{\partial x_1} & \frac{\partial g_1}{\partial x_2} & \cdots & \frac{\partial g_1}{\partial x_n} \\ \frac{\partial g_2}{\partial x_1} & \frac{\partial g_2}{\partial x_2} & \cdots & \frac{\partial g_2}{\partial x_n} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\partial g_m}{\partial x_1} & \frac{\partial g_m}{\partial x_2} & \cdots & \frac{\partial g_m}{\partial x_n} \end{pmatrix}$$

  - Orientation of tangent plane to the vector-valued function at a given point
  - Generalises the gradient of a scalar valued function
  - Linear Function
- Works well in practice for **moderate non-linearities**
- Large uncertainties in the beliefs leads to increased approximation errors

Robot Localisation

- **Localisation**: **Position and heading** of a mobile system in some reference frame
- Based on
  - Control information
  - Observations/sensor data
  - Maps (or some global frame)
- Estimate $p(\boldsymbol{x})$ for state $\boldsymbol{x}$
  - Often, we have

$$\textbf{2D:}\ \boldsymbol{x} = [x, y, \theta]^\top$$

$$\textbf{3D:}\ \boldsymbol{x} = \underbrace{[\underbrace{x, y, z}_{\text{“position”}}, \underbrace{\alpha_{roll}, \alpha_{pitch}, \alpha_{yaw}}_{\text{“heading”}}]^\top}_{\text{“pose”}}$$

- **Global Localisation vs. Tracking**

- **Global localisation**: Initially, the system can be anywhere
- **Pose tracking**: We know from where we start
- Important difference:
  - Types of Beliefs the system maintains
  - How much uncertainty can the localisation system handle?

Online vs. Offline Localisation

- All data available before computation
- Compute new pose whenever novel information becomes available

Sensor Odometry

- Compute incremental **motion updates** between time steps
- Use sensor data and not (only) wheel odometry
- Often obtained by registering sensor data sequential: pairs of images, pairs of point clouds, etc
- Provides relative motion estimates between different points in time based on sensor data
- Often, when there is no global map
- Not localisation but provides support

Visual (Inertial) Odometry

- Compute relative camera orientation between frames
- Scale from stereo camera or IMU
- Relative orientation plus scale provides 6DoF transformations
- Key ingredient: 5-point algorithm, RANSAC

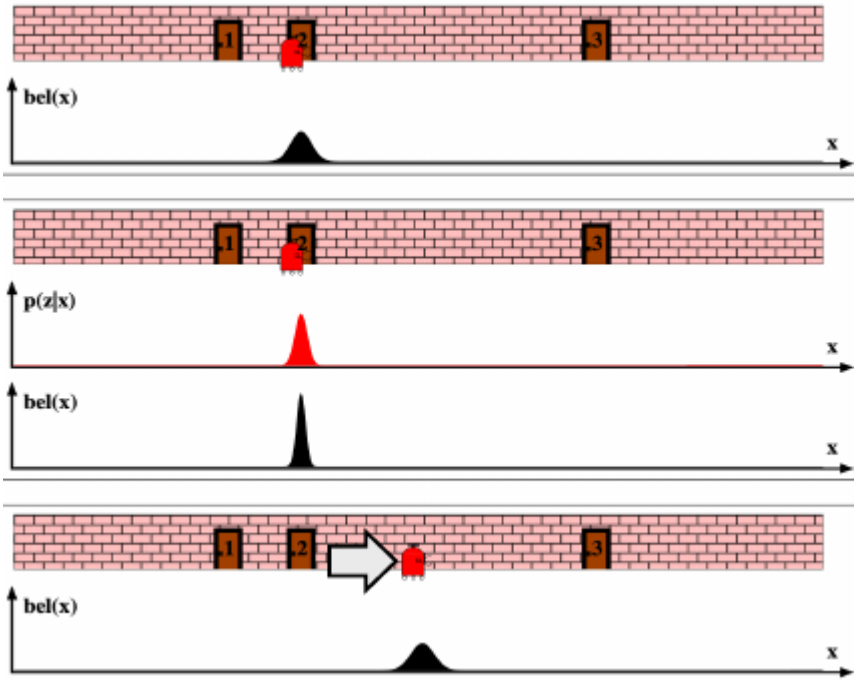
LiDAR Odometry/Incremental Scan Matching

- Register pairs or consecutive range scans
- Iterative Closest Point (ICP)
- Obtain 6 DoF transformations between pairs of recording poses

Key Approaches to Robot Localisation

- Markov localisation (grid localisation)
- Monte Carlo localisation
- Kalman filter localisation
- Least squares approaches
- Sliding window least squares

EKF Localisation



- Gaussian beliefs and measurements
- Often used to localise using landmarks
- Works well in practice for **moderate non-linearities**

1. **Algorithm EKF localization** $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t, c_t, m)$:
2. $\theta = \mu_{t-1,\theta}$
3. $G_t = \begin{pmatrix} 1 & 0 & -\delta_{\text{trans}} \sin(\theta + \delta_{\text{rot}_1}) \\ 0 & 1 & \delta_{\text{trans}} \cos(\theta + \delta_{\text{rot}_1}) \\ 0 & 0 & 1 \end{pmatrix}$ ← **Jacobian of g** $G = \frac{\partial g}{\partial x}$
4. $V_t = \begin{pmatrix} -\delta_{\text{trans}} \sin(\theta + \delta_{\text{rot}_1}) & \cos(\theta + \delta_{\text{rot}_1}) & 0 \\ \delta_{\text{trans}} \cos(\theta + \delta_{\text{rot}_1}) & \sin(\theta + \delta_{\text{rot}_1}) & 0 \\ 1 & 0 & 1 \end{pmatrix}$ ← **Control space to state space** $V = \frac{\partial g}{\partial u}$
5. $M_t = \begin{pmatrix} \alpha_1 \delta^2_{\text{rot}_1} + \alpha_2 \delta^2_{\text{trans}} & 0 & 0 \\ 0 & \alpha_3 \delta^2_{\text{trans}} + \alpha_4 (\delta^2_{\text{rot}_1} + \delta^2_{\text{rot}_2}) & 0 \\ 0 & 0 & \alpha_1 \delta^2_{\text{rot}_2} + \alpha_2 \delta^2_{\text{trans}} \end{pmatrix}$
6. $\bar{\mu}_t = \mu_{t-1} + \begin{pmatrix} \delta_{\text{trans}} \cos(\theta + \delta_{\text{rot}_1}) \\ \delta_{\text{trans}} \sin(\theta + \delta_{\text{rot}_1}) \\ \delta_{\text{rot}_1} + \delta_{\text{rot}_2} \end{pmatrix}$ ← **Prediction using g**
7. $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + V_t M_t V_t^T$ ← **Uncertainty of the prediction**

$Q_t = \begin{pmatrix} {\sigma_r}^2 & 0 \\ 0 & {\sigma_\phi}^2 \end{pmatrix}$

for all observed features $z_t^i = (r_t^i, \phi_t^i)^T$ **do**

  $j = c_t^i$

  $q = (m_{j,x} - \bar{\mu}_{t,x})^2 + (m_{j,y} - \bar{\mu}_{t,y})^2$

  $\hat{z}_t^i = \begin{pmatrix} \sqrt{q} \\ \text{atan2}(m_{j,y} - \bar{\mu}_{t,y}, m_{j,x} - \bar{\mu}_{t,x}) - \bar{\mu}_{t,\theta} \end{pmatrix}$ ← **Expected measurement**

  $H_t^i = \begin{pmatrix} -\frac{m_{j,x} - \bar{\mu}_{t,x}}{\sqrt{q}} & -\frac{m_{j,y} - \bar{\mu}_{t,y}}{\sqrt{q}} & 0 \\ \frac{m_{j,y} - \bar{\mu}_{t,y}}{q} & -\frac{m_{j,x} - \bar{\mu}_{t,x}}{q} & -1 \end{pmatrix}$ ← **Jacobian of h**

  $S_t^i = H_t^i \bar{\Sigma}_t {H_t^i}^T + Q_t$

  $K_t^i = \bar{\Sigma}_t [H_t^i]^T [S_t^i]^{-1}$

  $\bar{\mu}_t = \bar{\mu}_t + K_t^i (z_t - \hat{z}_t^i)$

  $\bar{\Sigma}_t = (I - K_t^i H_t^i) \bar{\Sigma}_t$

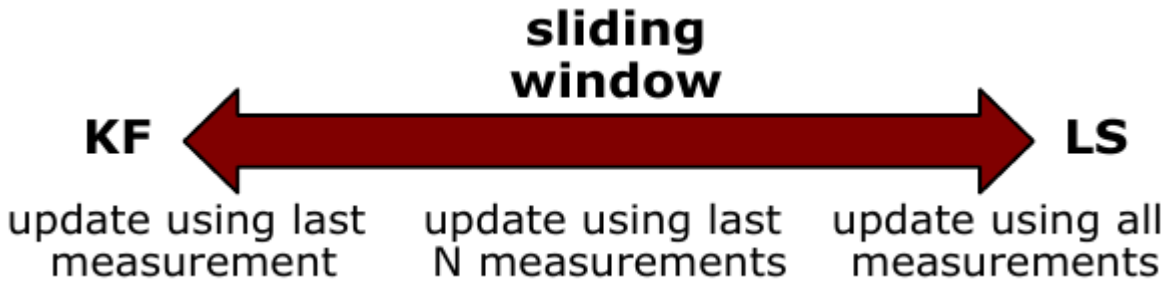
end for

$\mu_t = \bar{\mu}_t$

$\Sigma_t = \bar{\Sigma}_t$

- **return** μ_t, Σ_t

Localisation Using Least Squares

- Uses least squares and optimisation to compute the belief
- Offline approach, requires all data to be available beforehand
- Gaussian beliefs
- Often used as a references solution
- Least squares is traditionally and **offline** approach, not a Bayes filter
- Towards online localisation sliding-windows optimisations
- Approximations are often made
  - 

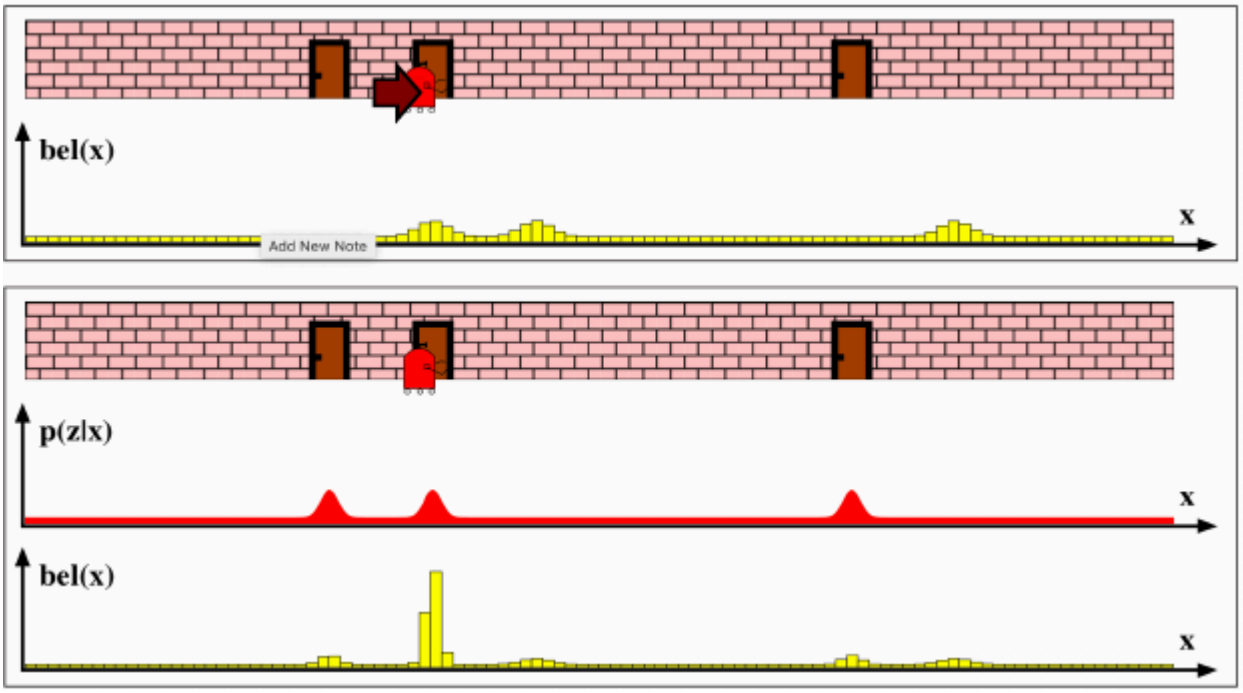
Non-Parametric Filters

- 2 implementations of the Bayes Filter
- **Non-parametric** filters
  - Realise probabilitstic localisation with **multimodal** distributions
  - Enable **global** localisation
  - Process **raw** range measurements
  - Use the motion and observation models presented in the previous chapter

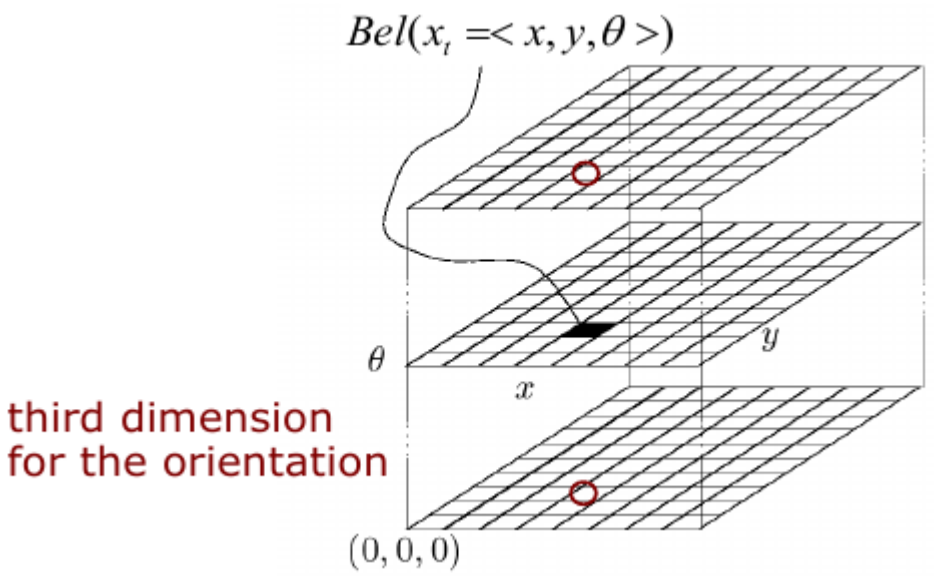
- **Multimodel Belief Distribution**
- **Represent** & **efficiently update** multimodel posterior?
    - **Discrete filter** (Grid/Markov Localisation)
    - **Particle filter** (Monte Carlo Localisation)

Markov Localisation

- Discretised belief about the state
- 3D or 6D histogram to model state
- Naturally models multi-model beliefs
- Discretisation limits pose accuracy
- All cells require updates for motions or observations



Piecewise Constant Representation



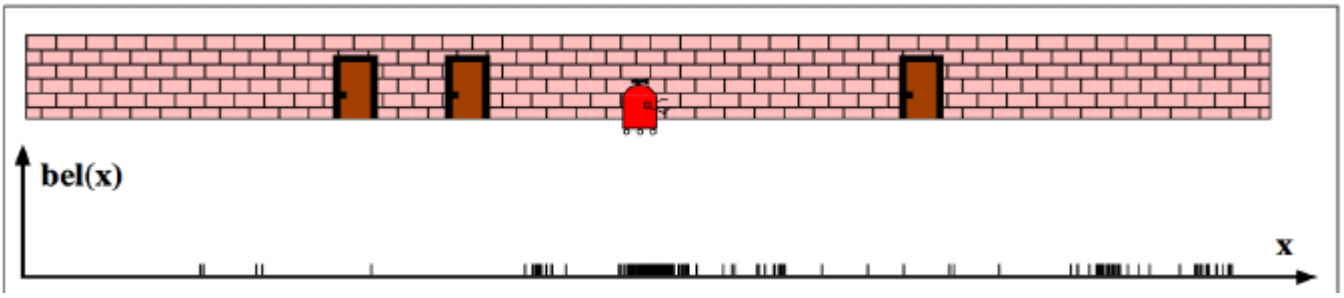
Processing Complexity reducing Techniques

- **Selective Updating** to Increase Efficiency
  - Update only relevant parts of state space
    - No update when probability under certain threshold
  - Update all, only if robot is **de-localised**
    - likelihood in "relevant" parts of state space "small"
- **Delayed Motion Updates**
  - Apply motion update at **lower frequency**
  - **Accumulate odometry readings** over a short period

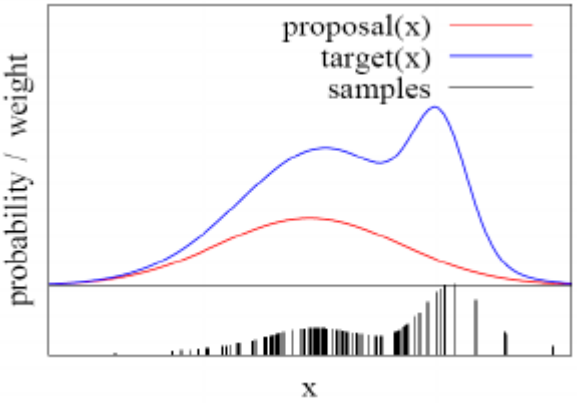
Summary

- Discretised belief about the state
- Histogram for representing density
- Naturally models **multi-modal** beliefs
- Performs **global localisation**, and recover from localisation errors
- Discretisation **limits pose accuracy**
- **All cell** require updates for motions or observations
- In practice: Approximations

Particle Filter



- Belief represented by **random samples**
- Sample = 3D or 6D state hypothesis
- Way to **efficiently** represent **non-Gaussian** distributions
  - Set of state hypotheses ("particles" or "samples")
  - Survival-of-the-fittest
- Models the distribution by set of weighted samples
- **Prediction**: sample from the proposal
- **Correction**: weigh particles by the ratio of target and proposal

- Sampling
  - Generate samples f (target)
    - from distribution π (proposal)
    - Pre-condition: $f(x) > 0 \rightarrow \pi(x) > 0$
    - **Weigth** for differences between π and f:
      - $w = \frac{f(x)}{\pi(x)}$
    - 

- **The more sample, the better the estimate**

Monte-Carlo Localisation

- **Each particle is a pose hypothesis**
- **Prediction**: For each particle, sample a new pose from the motion model (=proposal)
  - Odometry Motion Model:
    - Uncertainty in **translation & rotation** of the robot (Gaussian)
    - **For each particle**, draw a new pose by **sampling from three normal distribution**
- **Correction**: Weigh samples according to observation model (=target/proposal)
  - **Particle Weights** via Observation Model
    - Use beam-based model or end-point model
    - Compute **measurement likelyhood** by evaluating the actual observation w.r.t. the expected value
      - Assign it as **weight** of the particle
- **Resampling**: Draw sample i with probability $w_t^{[j]}$ and repeat j times (J=# particles)
  - Informally: "Replace unlikely samples by more likely ones"
    - Avoid too many samples in unlikely states
      - Only limited number of samples

current set of particles

Particle_filter($\mathcal{X}_{t-1}, u_t, z_t$):

$\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$

for $j = 1$ *to* J *do* — for all particles

  sample $x_t^{[j]} \sim p(x_t \mid u_t, x_{t-1}^{[j]})$ — prediction

  $w_t^{[j]} = p(z_t \mid x_t^{[j]})$ — correction

  $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[j]}, w_t^{[j]} \rangle$

endfor

for $j = 1$ *to* J *do* — resampling of particles

  draw $i \in 1, \ldots, J$ *with probability* $\propto w_t^{[i]}$

  add $x_t^{[i]}$ *to* $\mathcal{X}_t$

endfor

return $\mathcal{X}_t$ — new set of particles

- Particles propagated according to the motion model
- Particles weighed according to **likelyhood fo observation**
- Easy to implement and robust
- Used in many practical localisation systems
- **Based on appropriate motion and sensors models**
- Roulette wheel resampling easy to understand but **suboptimal in practice**
  - All samples have the same weight?

Low Variance Resampling: Implementation

```
1. draw random number r between 0 and 1/J

2. for (j=1 .. J)
        U = r + (j-1) * 1/J
        while (U > cum[i])  i++;
        pick_particle i
```

- Keeps samples in case of same weights
- Faster than roulette wheel resampling (O(J) vs. O(J log J))
- **Always use low variance resampling**

Localisation Error Recovery

- Error like Kidnapped robot problem
- At each time step randomly insert **fixed number of samples**

- OR, Insert number of random samples **proportional to average likelihood of particles**
- Without Random Sampling: No robot pose recovery